

플로이드 알고리즘: 모든 정점 쌍 최단 경로 핵꿀팁 정리

요요, 이번엔 플로이드 알고리즘에 대해 다시 한번 빠르게 정리해볼게. 이걸 그래프에서 모든 정점 쌍 사이의 최단 경로를 찾아내는 데 쓰는 존나 유용한 알고리즘이다.

1. 플로이드 알고리즘은 뭐하는 씹덕템이냐

- **목표:** 그래프 안에 있는 모든 정점(node) 쌍에 대해 제일 짧은 길을 찾아내. 마치 택배 배송 최적 경로를 한 번에 다 계산하는 거랑 비슷하다고 생각하면 된다.
- **어떻게 돌아가냐:** 3중 for 문으로 모든 정점을 거쳐가는 경우를 다 따져보면서 최단 경로를 계속 업데이트해. 마치 미로에서 모든 가능한 길을 다 탐색하는 거랑 비슷한 원리다.
- **다익스트라랑 뭐가 다르냐?** 이걸 중요하니까 확실히 알아둬라.
 - **다익스트라:** 특정 하나의 출발점에서 다른 모든 정점까지의 최단 경로를 찾을 때 써. 마치 내 집에서 다른 모든 장소까지 가장 빠른 길을 찾는 거랑 똑같다.
 - **플로이드:** 이걸 아예 모든 정점 쌍에 대한 최단 경로를 한방에 다 구해. 지도 앱에서 모든 도시 간의 예상 소요 시간을 한 번에 보는 거랑 같은 거다.

2. 초기값 설정 (인접 행렬)

플로이드 알고리즘을 쓰려면 먼저 인접 행렬이라는 걸로 그래프의 초기 상태를 설정해야 해. 이게 게임 시작 전 캐릭터 스탯 찍는 거랑 비슷한 거다.

- **$A[i][j]$ 는 뭔 뜻이냐:** 정점 i 에서 정점 j 로 가는 직접 연결된 간선(길)의 **비용(가중치)**을 나타내.
- **초기값 세팅법:**
 - **자기 자신한테 가는 길 ($i == j$):** 당연히 거리는 0이야.
 - **직접 연결된 길이 없는 경우:** 그 사이의 거리는 무한대로 설정해. 마치 존재하지 않는 길을 무한대의 비용으로 취급하는 거다.
 - **직접 연결된 길이 있는 경우:** 그 간선에 적힌 가중치를 그대로 써. 이게 바로 기본적으로 주어진 길의 비용이다.

3. 플로이드 알고리즘의 핵심: 경유 정점

플로이드 알고리즘의 핵심 아이디어는 ***어떤 두 정점 사이의 최단 경로는 직접 가는 길일 수도 있지만, 다른 정점을 하나 거쳐 가는 길이 더 짧을 수도 있다***는 거야. 여기서 그 "다른 정점"이 바로 **경유 정점**이다.

- **경유 정점 k :** 0번부터 마지막 정점($N-1$)까지, 모든 정점을 차례대로 임시 경유지로 삼아서 계산해. 마치 내비게이션이 여러 갈림길을 하나씩 탐색해보는 거랑 비슷하다.
- **경로 업데이트 조건:** 어떤 두 정점 i 에서 정점 j 로 가는 현재까지 알려진 최단 거리($A[i][j]$)와, 정점 k 를 거쳐서 가는 거리 ($A[i][k] + A[k][j]$)를 비교해.
 - **뜻:** 만약 k 를 거쳐 가는 거리가 현재 알고 있는 거리보다 더 짧으면, $A[i][j]$ 값을 그 더 짧은 거리로 업데이트하는 거야. 이걸 모든 i, j, k 조합에 대해 반복하는 거다.

4. 알고리즘 진행 과정 (단계별 최적화)

경유 정점 k 가 0부터 $N-1$ 까지 하나씩 증가하면서, 인접 행렬의 값들이 점점 최단 거리로 업데이트되는 과정을 이해하는 게 중요해.

- **$k = 0$ 일 때:** 0번 정점을 **잠재적인 모든 경로의 중간 지점**으로 고려해서 모든 i 에서 j 로 가는 경로를 확인하고 업데이트해.
- **$k = 1$ 일 때:** 이번엔 1번 정점을 **잠재적인 모든 경로의 중간 지점**으로 생각하고 똑같은 짓을 반복해.
- 이런 식으로 **k 가 모든 정점을 순회**하고 나면, 최종적으로 $A[i][j]$ 에는 i 에서 j 로 가는 **진짜 최단 거리**가 저장되게 돼. 마치 **여러 번의 탐색과 업데이트를 거쳐 미로의 모든 막힌 길을 제외하고 최적 경로만 남는 것과 같다.**

5. 플로이드 알고리즘 시간 복잡도

- **정점 N 개:** 플로이드 알고리즘은 3중 **for**문을 쓰기 때문에, 시간 복잡도는 무려 **$O(N^3)$** 이다. 정점 개수가 많아지면 계산량이 엄청나게 늘어난다는 걸 명심해라. 마치 **미로 크기가 세제곱으로 커지는 것과 같은 느낌**이다.

위상 정렬 (Topological Sort)

다음은 위상 정렬이야. 이건 **방향 그래프**에서만 의미가 있고, 정점들을 "먼저 해야 하는 순서"에 맞춰서 일렬로 나열하는 아주 특별한 정렬 방법이다.

1. 위상 정렬은 뭘 하는 신기한 정렬이냐

- **대상: 방향 그래프(Directed Graph)** - 간선에 방향이 있는 그래프에서만 의미가 있어. 마치 **일방통행 도로**로 이루어진 도시에서 길을 찾는 것과 비슷하다.
- **정의:** 방향 그래프의 모든 정점을 나열하되, **어떤 정점 u 에서 정점 v 로 가는 간선이 있다면, u 는 반드시 v 보다 먼저 나와야 해.** 마치 **선수 과목을 먼저 들어야 다음 과목을 들을 수 있는 학업 이수 순서**를 정하는 것과 같다.
- **주요 활용 분야:**
 - **선수 과목 이수 순서 결정:** 특정 과목을 듣기 전에 반드시 들어야 할 과목들을 순서대로 배열해.
 - **작업 의존성 관리:** 여러 작업이 서로 의존하는 관계가 있을 때, 어떤 작업부터 시작해야 하는지 결정해.
 - **빌드 시스템:** 소프트웨어 컴파일 순서를 정하거나, makefile 같은 빌드 스크립트에서 작업 순서를 정의하는 데 사용된다.

2. 위상 정렬의 필수 조건: 사이클(Cycle)이 없어야 한다

- 위상 정렬이 가능하려면 방향 그래프에 **사이클(Cycle), 즉 순환 구조가 없어야 해.** 마치 **시작도 끝도 없는 뫼비우스의 띠** 같은 구조에서는 순서를 정할 수 없는 것과 같다.
- **사이클이 있으면 왜 안 되냐:** 만약 A 를 먼저 해야 B 를 할 수 있고, B 를 먼저 해야 A 를 할 수 있는 사이클이 존재한다면, 도대체 뭘 먼저 해야 할지 결정할 수 없기 때문에 위상 정렬이 불가능해. 마치 **닭이 먼저냐 달걀이 먼저냐** 하는 끝없는 논쟁과 같은 상황이다.

3. 위상 정렬 알고리즘 (진입 차수 활용)

가장 흔하게 쓰이는 위상 정렬 알고리즘은 ****진입 차수(In-degree)****를 이용하는 방법이야. 진입 차수는 특정 정점으로 들어오는 간선의 개수를 의미해. 마치 **어떤 일을 하기 위해 먼저 완료되어야 하는 선행 작업의 수**라고 생각하면 된다.

• 알고리즘 단계:

1. **진입 차수 계산:** 그래프의 모든 정점에 대해 진입 차수를 계산해.
2. **진입 차수가 0인 정점 선택:** 진입 차수가 0인 정점들을 찾아. 이 정점들은 아무런 선행 조건이 없으므로 가장 먼저 처리할 수 있어. 마치 **가장 먼저 시작할 수 있는 독립적인 작업**과 같다.
3. **큐에 추가 및 출력:** 진입 차수가 0인 정점들을 큐에 넣고, 큐에서 꺼내면서 위상 정렬 결과에 추가해.
4. **간선 제거 및 진입 차수 업데이트:** 큐에서 꺼낸 정점에서 나가는 모든 간선을 제거해. 간선을 제거하면 그 간선이 가리키던 정점의 진입 차수가 1 감소해. 마치 **작업이 완료되면 그 작업에 의존하는 다른 작업들의 선행 조건 하나가 충족되는 것과 같다.**
5. **반복:** 다시 진입 차수가 0이 된 정점들을 찾아서 큐에 넣고 3~4단계를 반복해.
6. **종료:** 큐가 빌 때까지 이 과정을 반복해. 만약 큐가 비었는데 아직 모든 정점을 방문하지 못했다면, 그래프에 사이클이 존재한다는 뜻이다.

4. 위상 정렬 예시 (과목 이수 순서)

강의에서 제시된 과목 이수 순서 예시를 다시 한번 볼게.

과목명	코드 (정점)	선수 과목 (선행 정점)
컴퓨터개론	0	없음
이산수학	1	없음
C 언어	2	0 (컴퓨터개론)
자료구조	3	0, 1, 2
확률	4	1 (이산수학)
알고리즘	5	2, 3, 4

이 의존 관계를 방향 그래프로 나타내고 위상 정렬을 수행하면, 다음과 같은 가능한 이수 순서를 얻을 수 있어. (결과는 여러 가지가 나올 수 있다!) 예를 들어, **컴퓨터개론 -> 이산수학 -> C 언어 -> 확률 -> 자료구조 -> 알고리즘** 또는 **이산수학 -> 컴퓨터개론 -> C 언어 -> 확률 -> 자료구조 -> 알고리즘** 등이 가능해.

정렬 (Sorting)

마지막으로 정렬에 대한 기본적인 내용을 복습해볼게.

1. 정렬은 왜 하는 건데?

- **정의:** 데이터를 특정 기준(예: 크기, 알파벳 순서)에 따라 **순서대로 나열**하는 기본적인 알고리즘이야. 마치 **책을 제목 순서대로 정리**하거나 **시험 점수를 높은 순서대로 나열**하는 것과 같다.
- **중요성:** 컴퓨터 과학에서 가장 기본적이고 중요한 개념 중 하나야. 효율적인 자료 탐색, 데이터 분석 등 다양한 분야에서 핵심적인 역할을 해.

2. 정렬 방식의 종류

정렬 방식은 크게 내부 정렬과 외부 정렬로 나눌 수 있어.

- **내부 정렬 (Internal Sort):** 정렬할 데이터를 ****메인 메모리(RAM)****에 모두 올려놓고 정렬하는 방식이야. 데이터 양이 적을 때 효율적이지.
- **외부 정렬 (External Sort):** 정렬할 데이터가 너무 커서 **메인 메모리에 한 번에 다 올릴 수 없을 때** 사용하는 방식이야. 하드 디스크와 같은 외부 저장 장치를 사용하여 데이터를 여러 번 읽고 쓰면서 정렬해.

3. 정렬의 핵심: 키 (Key)

- **키(Key):** 데이터를 정렬할 때 **기준**이 되는 값이야. 예를 들어, 학생 객체를 이름순으로 정렬할 때 "이름"이 키가 돼. 마치 ****사전에서 단어를 찾을 때 "표제어"가 기준이 되는 것과 같다.**

시험 대비 꿀팁 (진짜 마지막이다!)

- **플로이드 알고리즘:**
 - 개념(모든 쌍 최단 경로)과 다익스트라 차이점을 확실히 이해해라.
 - 초기 인접 행렬 만드는 법 (0, 무한대, 가중치) 숙지!
 - **경유 정점 k 를 이용한 $A[i][j]$ 업데이트 식 ($A[i][j] = \min(A[i][j], A[i][k] + A[k][j])$) 완벽하게 이해하고 설명할 수 있어야 한다.** (교수님 왈: "이 부분 코드 빈칸 채우기 시험에 나온다!")
 - 시간 복잡도 $O(N^3)$ 잊지 마라.
- **위상 정렬:**
 - 방향 그래프, 사이클 없어야 함! 이거 두 개 무조건 기억해라.
 - 진입 차수 이용한 알고리즘 단계별 설명 가능해야 한다.
 - 선수 과목 예시 등으로 위상 정렬 필요성을 설명할 수 있어야 한다.
 - **위상 정렬 코드 (교재 예제) 흐름 파악해둬라.** (교수님 왈: "이거 손으로 따라 해보는 문제 낼 수도 있다!")
- **정렬:**
 - 정렬 기본 개념, 내부/외부 정렬 차이점, 키(Key)의 개념과 중요성 이해하면 된다.